Which LLM Serving Engine Should I Deploy?

An Empirical Comparison of vLLM, SGLang, and TensorRT-LLM Under Load

Anonymous Author(s)

ABSTRACT

Large language model (LLM) services are deployed on open-source serving engines whose schedulers decide, under load, which requests run, which wait, and which are evicted. These decisions are invisible to the application, yet they determine what users experience. Published comparisons of serving engines mostly report peak throughput or per-token speed, which says little about how an engine behaves once its memory or scheduling capacity is exhausted. We present an empirical comparison of three widely used engines, vLLM, SGLang, and TensorRT-LLM, driven through their own OpenAI-compatible frontends by a common open-loop benchmark harness. We measure six user-facing metrics (time to first token, time between tokens, time per output token, end-to-end latency, throughput, and request outcome) and combine them into SLO goodput, across three models with different key–value (KV) cache footprints, realistic mixed and multi-turn workloads, five arrival processes, two scheduler capacity settings, two prefill-handling features, n-gram speculative decoding, and two hardware platforms.

The engines are nearly indistinguishable at light load and diverge sharply once a resource saturates, and they diverge in *where* they make requests wait. On heterogeneous traffic the engine with the fastest token generation delivers the lowest goodput, while on uniform traffic it is the fastest end to end; a performance feature that helps one engine can hurt another. We also report methodology pitfalls that change conclusions: time to first token measured from dispatch hides client-side queueing, finite traces make latency non-monotonic in load, and identically named engine metrics are not comparable. We conclude with a decision guide for practitioners and release the harness and data. [TODO: tighten to the final RQ findings; ICPE abstracts are typically 150–250 words]

CCS CONCEPTS

• **Software and its engineering** → **Software performance**; • **General and reference** → *Experimentation*; • **Computing methodologies** → Artificial intelligence.

KEYWORDS

LLM inference serving, benchmarking, vLLM, SGLang, TensorRT-LLM, KV cache, scheduling, SLO goodput, tail latency

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICPE '27, 2027, TBD
© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Anonymous Author(s). 2027. Which LLM Serving Engine Should I Deploy?: An Empirical Comparison of vLLM, SGLang, and TensorRT-LLM Under Load. In *Proceedings of the 2027 ACM/SPEC International Conference on Performance Engineering (ICPE '27).* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Serving a large language model (LLM) is a queueing problem with an unusual resource: the key–value (KV) cache, which grows with every token a request generates and must stay in GPU memory until the request finishes. Modern serving engines—vLLM [12], SGLang [31], and TensorRT-LLM [19]—batch requests continuously at iteration granularity [30] and manage the KV cache in blocks, which lets them keep hundreds of requests in flight on one GPU. When the KV cache or the scheduler's slots run out, however, each engine must choose: admit a new request and risk evicting a running one, or hold the new request back. These choices are made inside the engine, are rarely documented as part of the engine's contract, and are not exposed through the OpenAI-compatible APIs that applications use.

Practitioners choosing an engine typically rely on published per-token throughput figures or on vendor benchmarks run at a fixed, comfortable load. Such figures answer the question "how fast can this engine generate tokens?" but not the question operators actually face: "what will my users experience when traffic is heavy, request lengths vary, and memory is tight?" The answer depends on several metrics at once—how long a user waits for the first token, whether the stream stalls midway, how long the whole answer takes, how many requests fail—and on which of these the application's service-level objective (SLO) is bound by.

This paper compares vLLM, SGLang, and TensorRT-LLM empirically under conditions that expose their scheduling behaviour. We drive all three through their own OpenAI-compatible serving frontends with a common open-loop harness, AnonBench, using realistic workloads assembled from public corpora, Poisson and bursty arrival processes, and three models whose per-token KV footprint differs by up to 6.25×, including one model with full multi-head attention (MHA) and two with grouped-query attention (GQA) [5]. We organise the study around four research questions:

- **RQ1** *Engine behaviour under load.* How do the engines compare in responsiveness, streaming quality, end-to-end latency, throughput, reliability, and SLO goodput as load and KV-cache pressure increase, and what scheduler behaviour explains the differences?
- **RQ2** *Prefill handling.* How do prefix caching and chunked prefill change latency under load on each engine?
- **RQ3** *Speculative decoding.* Does n-gram speculative decoding improve latency and throughput, and is the effect consistent across engines?

RQ4 *Hardware generalisation.* Do the **RQ1** findings hold on a different hardware platform with a much larger, unified memory?

Findings. [TODO: finalise against Section 4; the statements below reflect the current analysis]

- At light load the engines are interchangeable: decode speed differs by at most 6%. They diverge once a resource saturates, and the dominant difference is *where* each engine makes requests wait: vLLM admits early and later stalls running streams, SGLang holds requests before their first token and then streams smoothly, and TensorRT-LLM admits the fewest requests.
- The engine with the fastest token generation (TensorRT-LLM) has the lowest SLO goodput on heterogeneous traffic, yet the lowest end-to-end latency on uniform multi-turn traffic. Under the SLOs we study, goodput is decided by time to first token, not by token speed.
- Raising the scheduler's sequence limit doubles goodput when slots are the bottleneck and has no effect when the KV cache or the engine's admission policy is.
- Performance features are engine-specific: prefix caching helps all engines alike, whereas chunked prefill and n-gram speculative decoding help one engine and hurt another.
- The engine ranking under KV pressure is preserved on a second platform with 2.7× the memory and newer engine versions.

Contributions.

(1) A controlled, multi-metric comparison of three production LLM serving engines under load, covering latency, streaming quality, throughput, reliability, and SLO goodput (Section 4).
(2) An explanation of the observed differences in terms of each engine's admission and eviction behaviour, the scheduler's sequence limit, and the KV-cache footprint of the model.
(3) A set of methodology lessons for benchmarking LLM serving, each of which changes a conclusion if ignored (Section 5).
(4) A decision guide for practitioners, and an open benchmark harness with all per-request data (Section 8).

The remainder of the paper is organised as follows. Section 2 summarises the serving mechanisms that matter for our results. Section 3 describes the harness, engines, models, workloads, and metrics. Section 4 answers the research questions. Section 5 discusses implications for practitioners and for benchmarking, Section 6 threats to validity, and Section 7 related work.

2 BACKGROUND

Prefill, decode, and the KV cache. An autoregressive transformer [28] serves a request in two phases. The *prefill* phase processes the whole prompt in one forward pass and produces the first output token; the *decode* phase then generates one token per forward pass. Each layer stores the keys and values of every processed token in the KV cache, so a request's cache footprint grows linearly with its prompt and output length. The footprint per token is $2 \times L \times H_{kv} \times d_{head} \times b$ bytes for L layers, H_{kv} KV heads of dimension d_{head}, and b bytes per element. Multi-query [25] and grouped-query attention [5] reduce H_{kv} below the number of query heads; models with full multi-head attention (MHA) keep one KV head per query head and therefore need several times more cache per token.

Continuous batching and paged KV memory. Serving engines batch at iteration granularity: requests join and leave the running batch between forward passes [30]. To avoid fragmentation, vLLM's PagedAttention stores the KV cache in fixed-size blocks allocated on demand [12]; SGLang and TensorRT-LLM use comparable block-based managers. The number of requests that can run concurrently is bounded by two limits: a configured maximum number of running sequences (`max_num_seqs` in vLLM; analogous settings in the other engines) and the free capacity of the KV-cache pool.

Admission, preemption, and eviction. When the KV pool fills, an engine must either stop admitting requests or reclaim memory from running ones. Engines differ in how much memory they reserve when they admit a request. Allocating on demand admits the most requests but may later run out of memory mid-generation, forcing the engine to *preempt* a running sequence, discard its cache, and recompute it later. Reserving memory for a request's maximum output length up front avoids eviction but admits fewer requests. [TODO: document each engine's default policy and the configuration used here: vLLM V1 preempt-and-recompute; SGLang retraction with an estimated new-token ratio; TensorRT-LLM capacity-scheduler policy (verify against the launch log)]

Prefill-handling features. Prefix caching retains the KV cache of previously processed prompts so that a request sharing a prefix—for example, the next turn of a conversation—skips recomputing it; SGLang organises this cache as a radix tree [31]. *Chunked prefill* splits a long prompt into fixed-size chunks that are interleaved with decode steps, bounding how long a long prompt can stall the running batch [4]. Disaggregating prefill and decode onto different devices [20, 32] is out of scope for this single-GPU study.

Speculative decoding. Speculative decoding drafts several tokens cheaply and verifies them in one forward pass of the target model, accepting the longest correct prefix [9, 13]. *N-gram* (prompt-lookup) speculation drafts by matching the most recent tokens against earlier text in the context and needs no draft model [23]. Its benefit depends on the draft acceptance rate and on the extra work verification adds to each batch.

User-facing latency metrics. Time to first token (TTFT) is dominated by queueing and prefill; the time between streamed tokens (TBT) and the time per output token (TPOT) by decode; end-to-end latency (E2E) by both. Because no single metric captures user experience, recent work evaluates serving systems by *goodput*: the rate or share of requests that meet all SLO targets at once [32].

3 METHODOLOGY

3.1 Benchmark harness

AnonBench is an asynchronous Python client that drives each engine through its own OpenAI-compatible `/v1/completions` endpoint with streaming enabled, so every engine is measured through

Table 1: Engines and platforms.

	L40S (**RQ1–RQ3**)	GB10 (**RQ4**)
vLLM (V1)	0.11.0	0.32.0
SGLang	[VERIFY: version]	0.5.22
TensorRT-LLM (PyTorch)	1.0.0	1.2.1
Memory	44.4 GiB	121 GiB unified
KV memory fraction	0.90 [VERIFY:]	0.80

the interface applications use, including its HTTP frontend and tokenizer. For each request the harness records the dispatch time, the arrival time of every streamed chunk, the number of output tokens, and the final status. In parallel it polls each engine's native metrics endpoint (Prometheus `/metrics` for vLLM and SGLang, a JSON statistics endpoint for TensorRT-LLM) for KV-cache occupancy, running and waiting sequence counts, preemption counters, and prefix-cache hit rates.

Load generation. Arrival times are generated in advance and requests are released open-loop, independent of completions [24]. We use four Poisson processes (2, 4, 8, and 16 requests/s) and one bursty process (10 requests 50 ms apart, every 5 s). A client-side semaphore caps the number of in-flight requests (*client concurrency*, 16 or 256). For multi-turn conversations, a turn is released only after the previous turn of the same conversation completes, as a real user would; conversations are interleaved open-loop.

Isolation. Each rate configuration runs against a freshly launched server, so caches and counters start empty. [TODO: state warm-up policy and inter-run cool-down] Requests that receive no first token within 300 s, or do not finish within 600 s, are recorded as timeouts.

3.2 Engines

Table 1 lists the engines and versions. All engines serve fp16 weights on one GPU (tensor parallelism 1) with a maximum context of 8,192 tokens. The scheduler's sequence limit (`max_num_seqs`) is set to 64 or 256 and mapped to each engine's equivalent setting. [TODO: list the per-engine flags: KV memory fraction, maximum batched tokens, maximum running requests, maximum batch size, and the capacity-scheduler policy] Two engine-specific adjustments were necessary. First, TensorRT-LLM treats the chunk size as a hard prompt-length limit unless chunked prefill is enabled explicitly, so the chunked-prefill strategy sets this flag. Second, TensorRT-LLM's n-gram speculation is incompatible with its overlap scheduler, which we therefore disable whenever n-gram speculation is on; SGLang disables its equivalent optimisation automatically and caps running requests at 48 when speculation is on.

3.3 Models

We select three instruction-tuned models whose per-token KV footprint spans a wide range (Table 2). Qwen1.5-14B-Chat [8] is the only one without any KV-head reduction ($H_{kv} = 40$), so it needs 5× the cache per token of Mistral-NeMo-12B and 6.25× that of Llama-3.1-8B [16]. We considered distilled reasoning models but rejected them after confirming from their configuration files that they use GQA backbones and would not provide the architectural contrast.

Table 2: Models. KV bytes per token at fp16.

Model	Attn.	H_{kv}	Layers	KV B/token
Llama-3.1-8B-Instruct	GQA	8	32	131,072
Mistral-NeMo-12B-Instr. [18]	GQA	8	40	163,840
Qwen1.5-14B-Chat	MHA	40	40	819,200

Table 3: Workloads (per rate configuration; token counts are medians).

Workload	Task (source)	Req.	Prompt	Max out
`mixed_real`	chat (ShareGPT)	125	20	239
	code (MBPP)	125	53	45
	reasoning (GSM8K)	125	85	6,810
	summ. (CNN/DM)	125	791	64
`chat_multiturn`	2–6 turns (ShareGPT)	280–600	454	310
`chat_real_long`	≥500 tok. (ShareGPT)	300	938	[VERIFY:]

Prompt formatting. The harness sends plain instruction text to the completions endpoint. Qwen1.5-14B-Chat emits its end-of-turn token immediately on unformatted prompts, so for this model the harness applies the model's chat template at send time. [TODO: decide whether to template all models; see Section 6 on output length and zero-token replies]

3.4 Workloads

All workloads are built from public corpora (Table 3). `mixed_real` mixes four task types in equal shares: chat prompts from ShareGPT [1], code generation from MBPP [7], reasoning from GSM8K [10], and summarisation from CNN/DailyMail [11]. It combines short prompts with very long requested outputs (reasoning) and long prompts with short outputs (summarisation), so a few long-running requests can occupy scheduler slots and KV memory for minutes. `chat_multiturn` replays ShareGPT conversations of two to six turns; each turn resends the conversation history, which makes prefix reuse possible. `chat_real_long` uses single-turn ShareGPT prompts of at least 500 tokens to stress prefill.

3.5 Metrics

TTFT Time to first token. For open-loop `mixed_real` we measure it from the request's scheduled *arrival*, i.e., including any time spent queued at the client; measuring from dispatch hides that queueing (Section 5). For `chat_multiturn`, where later turns are deliberately held until the previous turn ends, we measure from dispatch.

TBT Time between consecutive streamed chunks, summarised per request by its 95th percentile and maximum (the longest stall).

TPOT Time per output token, $(\text{E2E} - \text{TTFT})/(n-1)$ for n output tokens. Unlike TBT it is unaffected by how many tokens a chunk carries, which matters under speculative decoding.

E2E End-to-end latency, from arrival to the last token.

Table 4: Experimental design.

RQ	Configurations
RQ1	3 models; `mixed_real`, `chat_multiturn`; client concurrency 16, 256 (256 only for Qwen1.5); `max_num_seqs` 64, 256; baseline strategy
RQ2	Mistral-NeMo; `chat_multiturn`, `chat_real_long`; prefix caching on/off × chunked prefill off/512 tokens; concurrency 256
RQ3	Mistral-NeMo; `mixed_real`, `chat_multiturn`; baseline vs. n-gram (k=3); `max_num_seqs` 32, 64; concurrency 256
RQ4	**RQ1** protocol on GB10 at concurrency 256

Throughput Output tokens per second over a rate configuration.

Outcome Finished, timed out, or *zero-token reply* (the server ends the stream without producing a token).

Goodput The share of *all* requests that finish while meeting both a TTFT and a TPOT target. We use a *strict* SLO (TTFT $\leq$ 2 s, TPOT $\leq$ 200 ms) and a *relaxed* SLO (TTFT $\leq$ 4 s, TPOT $\leq$ 500 ms).

Censoring. Discarding timed-out requests would make an overloaded engine look fast. We therefore count a request with no first token by 300 s as TTFT = 300 s and E2E $\geq$ 300 s, and a request that started but did not finish by 600 s as E2E = 600 s. Zero-token replies are excluded from latency statistics and count as goodput misses.

Scheduler-internal metrics. Engine-reported metrics are used only to explain user-facing results, because their semantics differ between engines: vLLM reports a cumulative prefix-cache hit rate and a per-server preemption counter; SGLang reports the hit rate of the most recent batch and, from version 0.5, a retraction counter; TensorRT-LLM's statistics endpoint is read once after each run in our harness.

3.6 Experimental design

Table 4 summarises the configurations per research question. Every configuration is run with all three engines and all five arrival processes.

In total the study comprises [TODO: count] rate configurations and [TODO: count] requests on the L40S and 180 rate configurations and 72,000 requests on the GB10. Each configuration is run once.

4 RESULTS

4.1 RQ1: Engine behaviour under load

[TODO: write from Flow C]

4.2 RQ2: Prefill handling

[TODO:]

4.3 RQ3: Speculative decoding

[TODO:]

Table 5: Engine choice by deployment situation.

Situation	Prefer	Basis
Uniform requests, steady load	TensorRT-LLM	**RQ1**
Mixed lengths, TTFT-bound SLO	vLLM	**RQ1**
Smooth streaming required	SGLang/ TensorRT-LLM	**RQ1**
Model barely fits in KV memory	vLLM, more memory	**RQ1**, **RQ4**
Slot-bound deployment	raise `max_num_seqs`	**RQ1**
Long prompts	chunked prefill on TensorRT-LLM only	**RQ2**
Multi-turn chat	prefix caching on any engine	**RQ2**
N-gram speculation	vLLM only	**RQ3**

4.4 RQ4: Hardware generalisation

[TODO:]

5 DISCUSSION

5.1 Choosing an engine

Table 5 condenses the results into guidance. Its central message is that the right engine depends on three properties of the deployment: how heterogeneous the request lengths are, which latency the SLO is bound by, and how much KV-cache headroom the model leaves. [TODO: finalise the table once Section 4 is written]

5.2 Lessons for benchmarking LLM serving

Several measurement choices that look innocuous changed our conclusions. We report them because they apply to any LLM serving benchmark.

Measure TTFT from arrival, not from dispatch. A client that caps in-flight requests and starts the TTFT clock at dispatch silently excludes the time requests spend queued at the client—the classic coordinated-omission problem [27]. In our data this hidden wait reached a 95th percentile of over 200 s for the most overloaded engine, so dispatch-based TTFT understated the gap between engines precisely where it was largest. Open-loop arrival-based measurement [24] avoids this; for closed-loop multi-turn workloads, dispatch-based TTFT is the correct choice.

Count timeouts; do not drop them. Discarding requests that time out removes an overloaded engine's worst latencies and can invert rankings. Censoring them at the timeout value and counting them as SLO misses keeps the comparison honest.

Finite traces make latency non-monotonic in load. With a fixed number of requests, a low arrival rate stretches the trace over minutes, so late arrivals meet a KV cache already full of long-running generations, while at a high rate all requests arrive before the cache fills. In our data the 95th-percentile TTFT of one configuration was 185 s at 2 requests/s and 0.3 s at 16 requests/s; the cost moved from the first token to mid-stream stalls. Reporting latency against position in the run, or using duration-based traces, exposes this effect.

Identically named engine metrics are not comparable. Engines report prefix-cache hit rates over different windows (cumulative, last batch, or once per run), and the gap between streamed chunks shrinks or grows with the number of tokens per chunk under speculative decoding. We therefore compare engines only on client-side metrics and use engine-reported values to explain them. TPOT, unlike TBT, is robust to multi-token chunks.

Prompt formatting affects load, not just quality. Sending unformatted prompts to instruction-tuned models changed how many tokens they generated and, for some models, produced replies with zero tokens. Both change the load an engine experiences; a benchmark should either apply each model's chat template or report output-length distributions alongside latency.

Per-token speed is a poor proxy for service quality. Under the SLOs we study, goodput was decided almost entirely by TTFT: requests that missed only the TPOT target were at most 2% in every configuration. Benchmarks that report tokens per second at a comfortable load measure the property that least often limits real deployments.

6 THREATS TO VALIDITY

Construct validity. Our goodput SLOs (2 s/200 ms and 4 s/500 ms) are representative of interactive chat but not of every application; a tighter TPOT target favours the engine with the fastest decode, and we report that variant in the supplementary material. Engine-reported metrics have different semantics across engines (Section 3.5); we use them only to explain client-side results. Prefix-cache hit rates are measured per server, not per request.

Internal validity. Configuration options are mapped to each engine on a best-effort basis; "the same" nominal setting (e.g., chunk size, sequence limit, n-gram window) reflects each engine's own implementation. Two configuration defects were found and fixed before the corresponding sweeps (Section 3.2); others may remain in less-exercised code paths. Enabling n-gram speculation on TensorRT-LLM required disabling its overlap scheduler, which confounds that engine's speculative-decoding result [TODO: add a baseline with the overlap scheduler disabled]. TensorRT-LLM's KV-cache and queue statistics were not observable live, so its scheduler behaviour is inferred from client-side measurements. [TODO: confirm its capacity-scheduler policy from the launch log]

Prompts are not chat-templated for Llama-3.1-8B and Mistral-NeMo-12B. Llama-3.1-8B consequently generates about six times longer reasoning outputs than the other models, raising its KV pressure on `mixed_real`, and Mistral-NeMo-12B returns zero-token replies for 7–17% of requests on every engine. Both effects are identical across engines and therefore do not bias engine comparisons within a model, but they limit comparisons across models.

External validity. Results come from single-GPU deployments of 8–14B-parameter models on two platforms, with fp16 weights and no tensor parallelism. Larger models, quantisation, multi-GPU deployments, and disaggregated serving may change both absolute numbers and rankings. The **RQ4** platform also uses newer engine versions, so hardware and software effects are not separated there. Arrival processes are synthetic, and real traffic may differ in burstiness and length mix.

Conclusion validity. Each configuration was run once, so we report no confidence intervals. We mitigate this by relying on effects that are large relative to run-to-run variation (often an order of magnitude), by observing them consistently across arrival rates and models, and by reproducing the main ranking on a second platform. [TODO: add repeated runs for a subset of configurations]

7 RELATED WORK

LLM serving systems. Orca introduced iteration-level scheduling [30]; vLLM added paged KV-cache management [12]; SGLang added prefix sharing through a radix-tree cache and a frontend language for structured programs [31]; TensorRT-LLM provides NVIDIA's optimised kernels and in-flight batching [19]. Research systems improve on specific aspects: chunked prefill to bound decode stalls [4], prefill–decode disaggregation [20, 32], preemptive and fair scheduling [29], live migration across instances [26], and KV-cache-centric architectures [21]. These papers evaluate their own design against baselines; we instead compare widely deployed engines as black boxes, through the interface applications use. A survey of serving techniques is given by Miao et al. [17].

Benchmarking and evaluating LLM inference. MLPerf Inference [22] defines scenarios and latency constraints for inference benchmarks but targets submitted, tuned systems rather than default deployments under realistic mixed traffic. Etalon [2] argues that TTFT, TBT and TPOT individually miss user-visible stalls and proposes a fluidity metric; Andes [15] defines quality of experience for streamed text. Vidur [3] simulates serving configurations to search for good deployments. Tools such as LLMPerf [6] measure latency and throughput of serving endpoints. Our study complements these with a controlled cross-engine comparison under saturation, an explanation of the differences in terms of admission and eviction behaviour, and a set of methodology pitfalls observed in practice.

Performance-engineering methodology. Open- and closed-loop load generators produce different conclusions about the same system [24], and latency recorders that measure from dispatch suffer from coordinated omission [27]. Little's law [14] relates queue length, arrival rate and residence time; we use it to explain when the scheduler's sequence limit becomes the bottleneck. Our methodology lessons (Section 5.2) apply these principles to LLM serving, where long, variable-length generations make them especially consequential.

[TODO: add recent empirical comparisons of serving engines (2025–2026) and position against them]

8 CONCLUSION

We compared vLLM, SGLang, and TensorRT-LLM under load across six user-facing metrics and SLO goodput, three models with different KV-cache footprints, realistic workloads, and two hardware platforms. The engines behave alike until a resource saturates; beyond that point they differ mainly in how they admit requests and where they make them wait, and those differences decide which engine meets a given SLO. No engine is best across workloads,

metrics, and features: the fastest decoder can deliver the lowest goodput, and a feature that helps one engine can hurt another. We hope the decision guide and the methodology lessons help practitioners choose and evaluate serving engines for their own traffic. [TODO: finalise against the results]

Data and artifact availability. The benchmark harness, configuration files, per-request measurements for all runs, and the scripts that produce every figure and table will be released as an archived artifact with a DOI. [TODO: Zenodo DOI; link withheld for double-anonymous review]

REFERENCES

[1] 2023. ShareGPT Conversations (ShareGPT_Vicuna_unfiltered). https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered.

[2] Amey Agrawal, Anmol Agarwal, Nitin Kedia, Jayashree Mohan, Souvik Kundu, Nipun Kwatra, Ramachandran Ramjee, and Alexey Tumanov. 2024. Etalon: Holistic Performance Evaluation Framework for LLM Inference Systems. arXiv:2407.07000

[3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems (MLSys)*.

[4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[5] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[6] Anyscale. 2024. LLMPerf. https://github.com/ray-project/llmperf.

[7] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732

[8] Jinze Bai et al. 2023. Qwen Technical Report. arXiv:2309.16609

[9] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318

[10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168

[11] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching Machines to Read and Comprehend. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ACM, 611–626. https://doi.org/10.1145/3600006.3613165

[13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

[14] John D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387.

[15] Jiachen Liu, Zhiyu Wu, Jae-Won Chung, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024. Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services. arXiv:2404.16283

[16] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. arXiv:2407.21783

[17] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2023. Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems. arXiv:2312.15234

[18] Mistral AI. 2024. Mistral NeMo. https://mistral.ai/news/mistral-nemo.

[19] NVIDIA. 2026. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM. Accessed 2026.

[20] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*.

[21] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*.

[22] Vijay Janapa Reddi et al. 2020. MLPerf Inference Benchmark. In *47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*.

[23] Apoorv Saxena. 2023. Prompt Lookup Decoding. https://github.com/apoorvumang/prompt-lookup-decoding.

[24] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. 2006. Open Versus Closed: A Cautionary Tale. In *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[25] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150

[26] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[27] Gil Tene. 2015. How NOT to Measure Latency. Talk, Strange Loop.

[28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[29] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920

[30] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 521–538.

[31] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[32] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.